

2.4.2. Algoritmos Baseados em Intervalos

Na Seção 2.3.4, foram descritas maneiras de extrair características de séries temporais. Contudo, alguns algoritmos trabalham com a extração de características de intervalos delimitados das séries, em vez de sua totalidade. Em geral, esses algoritmos constroem combinações de modelos nas quais cada modelo se ajusta a um ou mais intervalos de um conjunto de séries.

Um dos mais simples modelos baseados em intervalos é a *Time Series Forest* (TSF) [Deng et al. 2013]. A TSF constrói uma árvore de decisão para cada intervalo escolhido aleatoriamente, sendo que a média, o desvio padrão e o *slope* de cada intervalo são usados como as características de entrada de cada árvore.

O *Random Interval Spectral Ensemble* (RISE) [Lines et al. 2018] funciona de forma muito similar à TSF. Porém, as características extraídas de cada intervalo vêm do domínio da frequência, e não do tempo, como o *Power Spectrum* (PS) e a *Autocorrelation Function* (ACF).

Por fim, os mais recentes avanços em algoritmos baseados em intervalos são a *Canonical Interval Forest* (CIF) [Middlehurst et al. 2020] e a *Diverse Representation Canonical Interval Forest* (DrCIF) [Middlehurst et al. 2021]. A CIF funciona da mesma forma que a TSF, ou seja, construindo árvores de decisão baseadas em características extraídas de intervalos. Entretanto, são adicionadas às características extraídas na TSF os 22 atributos presentes no *catch22* (descrito na Seção 2.3.4), o que contribui para que o CIF tenha grande poder discriminatório em vários conjuntos de dados. A DrCIF, por sua vez, expande a gama de atributos extraídos ao considerar, para cada instância, a sua primeira derivada e o seu periodograma, além dela própria, como entradas para a extração.

2.4.3. Algoritmos Baseados em Dicionários

Os algoritmos baseados em dicionário tentam discretizar as séries em sequências de padrões (ou *palavras*), criando, assim, dicionários que representam a frequência observada de cada palavra nessas séries. Os modelos, por fim, discriminam as classes ou valores-alvo de um conjunto de séries ao comparar os dicionários gerados a partir de cada uma.

Um exemplo desse tipo de algoritmo é o *Bag of SFA Symbols* (BOSS) [Schäfer 2015], que consiste em uma sequência de diversas etapas. Primeiro, são extraídas janelas deslizantes das séries, isto é, são considerados trechos de tamanho fixo w que se movem uma observação à frente a cada iteração. Cada janela, então, passa por um processo chamado *Symbolic Fourier Approximation* (SFA) [Schäfer and Höggqvist 2012], no qual seus l primeiros coeficientes espectrais (obtidos da sua Transformada Discreta de Fourier) são discretizados em c compartimentos (ou *letras*) de forma que cada compartimento contenha quase o mesmo número de instâncias.

Após a SFA de cada janela, cada série se torna uma sequência de palavras de l letras, formadas a partir de um alfabeto de tamanho c . Uma redução de redundância é então aplicada a essas sequências, ou seja, em caso de palavras iguais consecutivas, apenas o primeiro caso é considerado. O número de ocorrências de cada palavra é, então, contado em cada sequência, obtendo-se assim um conjunto de histogramas, dicionários ou sacola de palavras (do inglês, *bag of words*).

Por fim, a classificação de instâncias é feita por meio de um algoritmo de 1-Vizinho Mais Próximo, que considera os dicionários obtidos. A medida de distância considerada por ele é definida na Equação 3, chamada de *Distância BOSS* pelos autores originais. Note que, nesta medida, apenas as palavras não ausentes na primeira sacola são consideradas, tornando a distância não comutativa. Uma instância recebe, por fim, a classe da instância rotulada mais próxima.

$$\text{dist}(B_1, B_2) = \sum_{w \in B_1 | B_1(w) > 0} [B_1(w) - B_2(w)]^2 \quad (3)$$

O processo de transformação em histogramas realizado pelo BOSS é ilustrado na Figura 2.14.

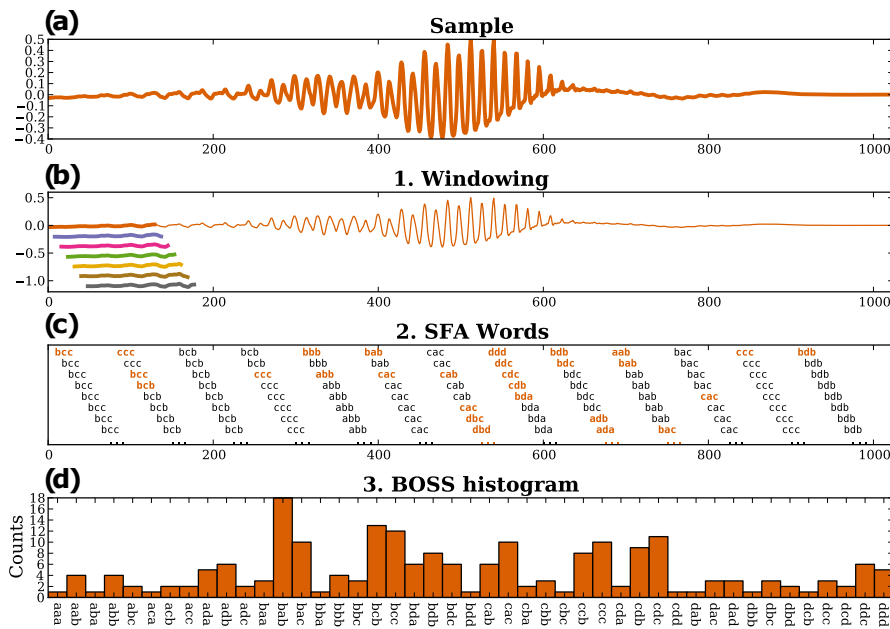


Figura 2.14. Ilustração das etapas do algoritmo BOSS utilizando-se palavras de tamanho 3 e alfabeto de tamanho 4. As palavras em preto são desconsideradas devido à redução de numerosidade [Schäfer 2015].

2.4.4. Aprendizado Profundo

O aprendizado profundo, ou em inglês *deep learning*, têm recentemente recebido grande atenção na literatura. Essa categoria de algoritmos de Aprendizado de Máquina tem ultrapassado o desempenho de outros algoritmos mais clássicos em diversas tarefas. Nesta seção, serão explicados os conceitos básicos para o entendimento em alto nível do aprendizado profundo e de redes neurais.

Esta categoria de algoritmos inspirou-se na estrutura de funcionamento do cérebro humano, englobando técnicas de treinamento de redes neurais para realizar tarefas complexas envolvendo processamento massivo de dados. Essas redes neurais consistem em camadas de funções conectadas que, assim como neurônios naturais, trocam informação entre si, mas processam seu sinal de entrada individualmente. Ou seja, este conceito pode ser entendido superficialmente como um modelo simplificado do cérebro humano.

Assim como o cérebro trabalha com as conexões entre os neurônios para desempenhar tarefas complexas, as redes neurais (artificiais) simulam essas conexões para capturar padrões nos dados por meio de pesos atribuídos às suas entradas, assim como as sinapses do cérebro humano.

Na Figura 2.15, é possível observar uma rede neural de exemplo, na qual uma série temporal de eletrocardiografia é tomada como entrada. Nessa representação, os neurônios ativados estão na cor vermelha. Na saída, é possível observar que o neurônio ativado com maior valor é o que indica a classe normal, ou seja, o ECG de entrada é pertencente a um paciente saudável, sem nenhuma anomalia no ritmo cardíaco.

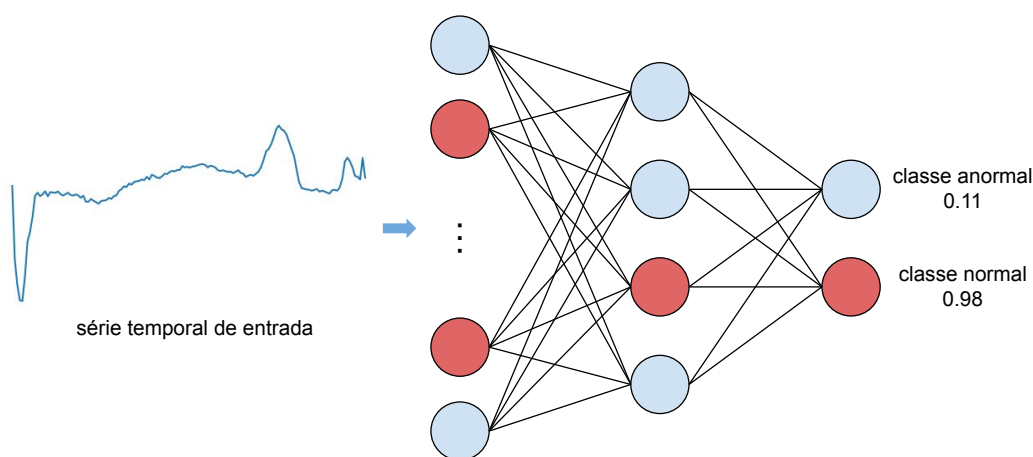


Figura 2.15. Exemplo de uma rede neural artificial na qual a entrada é uma série temporal de um eletrocardiograma e a saída são as classes normal e anormal, que indica uma anomalia.

Para o treinamento destas redes, faz-se necessária a existência de um conjunto de dados rotulados. Desta forma, a rede irá aprender as relações importantes nos dados para que o rótulo seja predito. Sejam essas relações importantes para definir um dado como pertencente a uma classe, como uma anomalia em batimento cardíaco pode indicar uma doença, ou uma relação em uma fotopletismografia indicando um valor de saturação do oxigênio, em uma tarefa de regressão extrínseca. Entretanto, é necessário um grande volume de dados para que a rede possa aprender essas relações complexas nos dados, o que pode ser um fator limitante para o uso de redes neurais.

Tratando-se especificamente de redes neurais no domínio de séries temporais, estudos comprovam a competitividade dessas técnicas quando comparadas com métodos do estado da arte, tanto para classificação quanto para regressão extrínseca [Fawaz et al. 2019, Foumani et al. 2023]. Dentre as arquiteturas de redes neurais utilizadas para obter bons desempenhos reportados na literatura, destaca-se o InceptionTime [Fawaz et al. 2020], que demonstrou uma boa capacidade de obter um bom desempenho para séries temporais de variados domínios.

2.4.5. Algoritmos Baseados em Comitês

Com esses e muitos outros algoritmos, a área de Aprendizado de Máquina para séries temporais também se enquadra em um clássico problema do Aprendizado de Máquina: o teorema “não há almoço grátis” [Wolpert 2002]. Como cada algoritmo compreende um viés indutivo diferente, não há um algoritmo que melhor desempenhe em quaisquer conjuntos de dados. Comparações experimentais em larga escala [Bagnall et al. 2017] podem mostrar que um algoritmo A é melhor que B na média dos resultados para os conjuntos de dados utilizados no experimento, mas sempre há uma fração dos conjuntos de dados nos quais B desempenha melhor que A .

Os vieses intrínsecos a modelos de Aprendizado de Máquina podem ser reduzidos combinando-os em modelos coletivos. Ao ajustar diferentes modelos suficientemente variados a um conjunto de dados e rotular uma instância levando-se em conta o voto de cada um, é possível obter um desempenho melhor do que aquele obtido com cada modelo individual. Essa estratégia é comumente referenciada como comitê.

Um dos primeiros modelos de comitê para classificação de séries temporais foi o *Collective of Transformation-Based Ensembles* (COTE, ou Flat-COTE) [Bagnall et al. 2015], que combinou modelos construídos a partir de diversos domínios das séries temporais: tempo, frequência, mudança e *shapelets*. Pouco adiante, foi proposto o *Hierarchical Vote Collective of Transformation-based Ensembles* (HIVE-COTE) [Lines et al. 2018], que melhorou o Flat-COTE ao adicionar uma estrutura hierárquica com votos probabilísticos, dois novos classificadores nos domínios pré-existentes e modelos em dois novos domínios. Os domínios considerados pelo HIVE-COTE são ilustrados por séries simuladas na Figura 2.16.

Por fim, o mais recente HIVE-COTE 2.0 [Middlehurst et al. 2021] substituiu dois classificadores já existentes e adicionou um novo, melhorando ainda mais o desempenho do algoritmo nos conjuntos de dados disponíveis.

2.5. Exemplos de Tarefas de Classificação e Regressão Extrínseca

Esta seção apresenta alguns exemplos práticos de classificação e regressão extrínseca de séries temporais utilizando as ferramentas `tsai` e `aeon`.

2.5.1. Conjunto de Dados

Neste capítulo, foram utilizados diferentes conjuntos de dados nos exemplos exibidos nas seções anteriores. Esses conjuntos de dados foram propostos para as tarefas de classificação e regressão extrínseca. Dois deles serão utilizados para realizar demonstrações práticas. Vale ressaltar que optamos por utilizar conjuntos de séries temporais já segmentadas e pré-processadas. No entanto, recomendamos ao leitor a exploração do repositório de dados PhysioBank³, que possui uma grande quantidade e diversidade de dados, majoritariamente séries temporais, provenientes da área da Saúde.

Para a tarefa de classificação, será utilizado o conjunto de dados ECG5000, disponibilizado no repositório *UCR Time Series Classification Archive* [Dau et al. 2018]. O conjunto é composto por 5000 trechos de eletrocardiogramas, das quais são encontradas

³<https://archive.physionet.org/physiobank/database/>

500 séries no conjunto de treino e 4500 séries no conjunto de teste. Cada série é univariada, possui 140 observações e pode estar rotulada em uma das 5 classes do conjunto. Neste contexto, esse conjunto simula um cenário onde a existência de instâncias rotuladas é limitada se comparado ao número de instâncias não rotuladas.

A Figura 2.17 mostra um exemplo de série para cada uma das classes contidas no conjunto ECG5000.

Os conjuntos utilizados na tarefa de regressão são: BIDMCRR, BIDMCHR e BIDMCSpO2, disponibilizados pelo repositório de dados para regressão extrínseca de séries temporais mantido pela universidade de Monash, a UEA e a UCR [Tan et al. 2021]. Os conjuntos dados BIDMCHR e BIDMCSpO2 dividem-se em 5550 instâncias para o conjunto de treino e 2399 instâncias para o conjunto de teste. Já o BIDMCRR, apesar de possui o mesmo número de instâncias no conjunto de teste, compreende 5471 instâncias para treino.

Todos os conjuntos são compostos por séries temporais multivariadas de 4000 observações. As duas variáveis observadas para esses conjuntos de dados são eletrocardiograma e fotopletismografia. A maior diferença entre esses conjuntos de dados se refere ao atributo alvo, sendo a frequência respiratória (BIDMCRR), a frequência cardíaca (BIDMCHR) e a oxigenação no sangue (BIDMCSpO2).

As Figuras 2.18, 2.19 e 2.20 mostram exemplos de instâncias dos conjuntos BIDMCHR, BIDMCRR e BIDMCSpO2, respectivamente. Cada uma das instâncias está acompanhada de seu valor alvo, sendo que foram selecionados exemplos cujos valores para os atributos-alvo fossem bastante distintos.

2.5.2. Classificação

A classificação de séries temporais é a tarefa de maior foco neste capítulo. A tarefa de classificação pode ser definida, em alto nível, como a indução de um modelo capaz de inferir rótulos para novas observações de um fenômeno. Por exemplo, considere um eletrocardiograma com anotações relativas à apneia. A tarefa de classificação consiste em induzir um modelo a partir desse conjunto de dados anotados e, para um novo paciente, utilizá-lo para inferir, a cada batimento, a presença ou ausência de apneia.

Historicamente, a classificação de séries temporais por meio de algoritmos muito simples se mostrou muito eficiente e eficaz. Como exemplo, o simples algoritmo do vizinho mais próximo foi utilizado com sucesso em uma ampla gama de aplicações. Na última década, inúmeros algoritmos surgiram para essa tarefa. A literatura de classificação de séries temporais tem buscado categorizar esses métodos e entende-se que há ao menos quatro categorias principais de algoritmos: baseados em distância, intervalos, dicionários e aprendizado profundo, já descritos na Seção 2.4. Além disso, há algoritmos que usam ideias contidas em mais de uma dessas categorias, além de algoritmos baseados em comitês de classificadores [Middlehurst et al. 2021].

É importante retomar o fato que há um compromisso entre custo computacional e eficácia de cada algoritmo [Ruiz et al. 2020]. Somado a isso, considera-se o teorema do “não há almoço grátis” [Wolpert 2002]. Sendo assim, é necessário conhecer técnicas baseadas em estimar o desempenho de diferentes algoritmos utilizando o conjunto de

treinamento, para ser decidido qual algoritmo utilizar nas próximas etapas. No exemplo dado a seguir, será respeitada a divisão de treino e teste do conjunto de dados. Assim, o classificador será induzido apenas com o conjunto de treino e será avaliado apenas com o conjunto de teste.

A seguir serão mostrados exemplos práticos de como utilizar cada um destes algoritmos com o conjunto de dados ECG5000.

O Código-Fonte 2.5 mostra um exemplo de aplicação do algoritmo baseado em distância do vizinho mais próximo. Neste caso, se aplica o algoritmo disponível na biblioteca `aeon` no conjunto de treino, instanciando-se os parâmetros para a classificação. Assim, quando se realiza uma predição no conjunto de testes, cada instância não rotulada tem sua similaridade calculada com base nas instâncias de treino. Baseando-se nas distâncias para um número de vizinhos estipulado pelo parâmetro k , o algoritmo vai determinar a qual rótulo aquela instância de teste pertence.

Código-Fonte 2.5. Classificação com o Algoritmo KNN.

```

1  from aeon.classification.distance_based import
    KNeighborsTimeSeriesClassifier
2  import numpy as np
3
4  ecg5000_train = np.genfromtxt('ECG5000_TRAIN.tsv',
    delimiter='\t')
5  ecg5000_test = np.genfromtxt('ECG5000_TEST.tsv',
    delimiter='\t')
6
7  x_train, y_train = ecg5000_train[1:], ecg5000_train[0]
8  x_test, y_test = ecg5000_test[1:], ecg5000_test[0]
9
10 knn = KNeighborsTimeSeriesClassifier()
11 knn.fit(x_train, y_train)
12
13 y_pred = knn.predict(x_test)
14
15 print(accuracy_score(y_test, y_pred))

```

Ao final do código é possível notar o cálculo da acurácia, que determina quantas instâncias de cada classe o algoritmo previu corretamente. A Figura 2.21 mostra a matriz de confusão para o algoritmo KNN no conjunto de dados. Nela é possível observar cada rótulo que o algoritmo previu corretamente, e nos casos de erro qual foi o rótulo atribuído pelo algoritmo para aquela instância no lugar do seu rótulo real. Como ocorre um desbalanceamento de classes neste conjunto, onde a maioria das classes são pertencentes aos rótulos 1 e 2, as outras classes que possuem poucos exemplos concentram grande parte dos erros do algoritmo. Desta forma, há pouco volume de instâncias nestas classes para o algoritmo aprender alguma espécie de padrão que permita uma predição correta.

No Código-Fonte 2.6 é possível observar a aplicação do modelo de aprendizado profundo InceptionTime para classificação no conjunto de dados ECG5000. A implementação baseia-se na utilização da biblioteca `tsai` [Oguiza 2022]. Note que com poucas

linhas de código é possível aplicar um algoritmo do estado-da-arte para classificação de séries temporais. O InceptionTime baseia-se em *InceptionBlocks* para capturar padrões e relações lineares e não lineares na estrutura das séries temporais. Contudo para utilizá-lo é preciso transformar as séries que normalmente são carregadas em um *array* de duas dimensões para três dimensões, sendo elas: o número de instâncias, o número de canais e a última dimensão o tamanho da série ou a quantidade de observações.

Código-Fonte 2.6. Classificação com o InceptionTime.

```

1 from tsai.all import *
2 import numpy as np
3
4 ecg5000_train = np.genfromtxt('ECG5000_TRAIN.tsv',
5                               delimiter='\t')
6 ecg5000_test = np.genfromtxt('ECG5000_TEST.tsv',
7                               delimiter='\t')
8
9
10 x_train, y_train = ecg5000_train[1:], ecg5000_train[0]
11 x_test, y_test = ecg5000_test[1:], ecg5000_test[0]
12
13 x_train = np.reshape(x_train, (x_train.shape[0], 1,
14                                x_train.shape[1]))
15 x_test = np.reshape(x_test, (x_test.shape[0], 1,
16                               x_test.shape[1]))
17
18 train_ds = TSDataset(x_train, y_train - 1, types=(TSTensor,
19                                                    TSLabelTensor))
20 train_dl = DataLoader(train_ds, bs=128, num_workers=0)
21
22 test_ds = TSDataset(x_test, y_test - 1, types=(TSTensor,
23                                                  TSLabelTensor))
24 test_dl = DataLoader(test_ds, bs=128, num_workers=0)
25 dls = DataLoaders(train_dl, test_dl,
26                   device=default_device())
27
28 num_classes = len(np.unique(y_train))
29 model = InceptionTime(1, num_classes)
30
31 learn = Learner(dls, model, loss_func=nn.CrossEntropyLoss(),
32               metrics=accuracy)
33 learn.fit_one_cycle(25, lr_max=1e-2)
34
35 preds, labels = learn.get_preds(dl=test_dl)
36 print(accuracy_score(preds.argmax(dim=1).numpy(),
37                      labels.numpy()))

```

A Figura 2.22 mostra a matriz de confusão resultante da predição do InceptionTime realizada nos dados de teste do conjunto ECG5000. Assim como na matriz de confusão do KNN, é possível notar que o comportamento é semelhante. Com métricas muito

semelhantes, necessita-se uma análise de outros fatores para a escolha dos modelos, tais como: custo computacional, volume de dados necessários, arquitetura da solução, entre outros fatores externos.

No caso da comparação entre o algoritmo de vizinho mais próximo e o InceptionTime, sempre que uma nova instância é inserida para a predição, o KNN precisa calcular todas as distâncias desta nova instância e das existentes para determinar o seu rótulo. Essa tarefa por muitas vezes pode ser custosa e tornar-se computacionalmente inviável em grandes volumes de dados. Porém, no caso do InceptionTime, o maior custo computacional está no treinamento da rede neural, uma vez treinada, para cada instância serão somente utilizados os pesos internos da rede para determinar seu rótulo, eliminando a necessidade de recalculas as distâncias. Desta forma, mesmo com métricas e resultados semelhantes, é possível determinar um modelo que melhor se adéqua ao caso proposto, contudo vale lembrar que não existe um modelo melhor para todos os casos.

2.5.3. Regressão extrínseca

Com o crescente interesse em séries temporais nas últimas décadas, que vem se intensificando nos últimos anos, pesquisadores propuseram dezenas de algoritmos específicos para a classificação [Ruiz et al. 2020] e outras tarefas, como *forecasting*, de séries temporais. Por outro lado, há ainda desafios não superados pela literatura para este tipo de dados. Um exemplo se dá pela lacuna por diferentes tarefas de aprendizado, como a regressão extrínseca de séries temporais. Por “extrínseca”, entende-se que o alvo da predição é um valor externo à série, ao contrário do que acontece com o *forecasting*. Por exemplo, ao considerar séries temporais de fotopletismografia, a regressão extrínseca teria por objetivo estimar parâmetros clínicos, como a saturação de oxigênio no sangue ou concentração de hemoglobina.

Recentemente, um grupo de pesquisadores notou esse fato e constatou que a tarefa de regressão é uma necessidade em diversos domínios de aplicação [Tan et al. 2021]. Dessa forma, criaram um repositório de conjuntos de dados de regressão em séries temporais e realizaram experimentos iniciais com diversos algoritmos⁴. A partir dessa iniciativa, foram propostas diversas novas técnicas para essa tarefa, incluindo algoritmos tradicionais aplicados a características extraídas das séries [Gay et al. 2021], adaptações de algoritmos bem estabelecidos em classificação [Guijo-Rubio et al. 2023] e de arquiteturas de redes neurais [Foumani et al. 2023].

Alguns algoritmos de classificação de séries temporais podem ser convertidos para lidar com regressão extrínseca de forma relativamente simples. Isso se deve ao fato de que muitos desses algoritmos consistem de uma ou mais etapas de transformação seguidas de um classificador final nos dados transformados.

Esse é o caso de um dos algoritmos mais bem avaliados em regressão extrínseca: o Random Convolutional Kernel Transform (ROCKET). O ROCKET utiliza um grande número de filtros convolucionais (em geral, 10000), construídos aleatoriamente a partir de diversos parâmetros, aplicando-os às séries temporais e extraindo características da série resultante, por padrão, valor máxima (max) e proporção de valores positivos (ppv). Essas

⁴<http://tseregression.org/>

características formam uma grande tabela atributo-valor, que é então usada como entrada para um classificador linear.

Para adaptar o ROCKET à regressão extrínseca, basta substituir o classificador linear na ponta final por um regressor linear. No Código-Fonte 2.7, por exemplo, mostra-se a utilização de um regressor ROCKET utilizando a biblioteca *aeon* para a predição de frequência cardíaca por meio do conjunto de dados BIDMCHR.

Código-Fonte 2.7. Regressão com o ROCKET.

```

1 from aeon.datasets import load_from_tsfile
2 from aeon.transformations.panel.rocket import Rocket
3
4 from sklearn.pipeline import Pipeline
5 from sklearn.linear_model import RidgeCV
6 from sklearn.metrics import mean_squared_error
7
8 X_train, y_train = load_from_tsfile('BIDMCHR_TRAIN.ts')
9 X_test, y_test = load_from_tsfile('BIDMCHR_TEST.ts')
10
11 model = Pipeline([
12     ('transformer', Rocket()),
13     ('regressor', RidgeCV())
14 ])
15 model.fit(X_train, y_train)
16 y_pred = model.predict(X_test)
17
18 print(mean_squared_error(y_test, y_pred, squared=False))

```

Ao final, o Código-Fonte imprime o valor do erro quadrático médio entre os valores observados e os valores preditos. A título de reforçar o exemplo da utilização da biblioteca *aeon* para regressão extrínseca, o Código-Fonte 2.8 mostra como realizar a predição de valores de frequência respiratória por meio do conjunto de dados BIDMCRR.

Código-Fonte 2.8. Exemplo de utilização da biblioteca Aeon.

```

1 from aeon.datasets import load_from_tsfile
2 from aeon.regression.distance_based import \
3     KNeighborsTimeSeriesRegressor
4
5 # Carregar o dataset para a memoria
6 X_train, y_train = load_from_tsfile('BIDMCRR_TRAIN.ts')
7 X_test, y_test = load_from_tsfile('BIDMCRR_TEST.ts')
8
9 # Definir o modelo
10 model = KNeighborsTimeSeriesRegressor(
11     distance='dtw',
12     n_neighbors=3
13 )
14
15 # Ajustar o modelo aos dados

```

```

16 model.fit(X_train, y_train)
17
18 # Obter as previsões
19 y_pred = model.predict(X_test)
20
21 print(mean_squared_error(y_test, y_pred, squared=False))

```

2.6. Considerações Finais

A área de Aprendizado de Máquina para séries temporais está em plena expansão. Graças ao surgimento de novas demandas e tecnologias na área da Saúde, como saúde móvel (*mHealth*), esse domínio de conhecimento tem sido um dos principais motivadores para esse crescimento. Para que essas aplicações em Saúde possam estar cada vez mais presentes no nosso dia-a-dia, ainda é necessário trabalhar em propostas e avaliação tanto de novas técnicas quanto novas aplicações.

No entanto, o estado-da-arte em classificação e regressão extrínseca de séries temporais enfrenta um problema relacionado ao compromisso entre eficiência e eficácia. Os estudos experimentais mais recentes para essas tarefas apontam para o fato que os algoritmos mais precisos são também os mais custosos.

Ao se utilizar séries temporais como medições de sinais para auxiliar médicos e pacientes com os cuidados da saúde, é preciso que as previsões realizadas pelos modelos de Aprendizado de Máquina sejam corretas. Por isso, é necessário se investigar por algoritmos que sejam menos custosos e possam fornecer respostas iguais ou melhores do que os de maior custo.

Uma tendência na literatura recente da área é o aparecimento de novas propostas no contexto de aprendizado profundo. Porém, muitas das arquiteturas investigadas nesse domínio são adaptações diretas de arquiteturas projetadas para outros tipos de dados, como imagens e vídeos. Para avançar significativamente na criação de modelos neurais para séries temporais, é necessário investigar por técnicas específicas para esse tipo de dados.

Ainda, uma vez que há um custo considerável para a obtenção de dados rotulados no domínio da Saúde, há um grande espaço para se pesquisar por técnicas que trabalhem com diferentes suposições de rótulos. Por exemplo, a criação de modelos a partir de pouco volume de dados, de aprendizado semissupervisionado ou autossupervisionado, aprendizado de uma classe, entre outros.

Por fim, os autores gostariam de agradecer ao apoio a este trabalho. O desenvolvimento deste documento só foi possível graças aos auxílios relacionados aos processos nº 2022/03176-1, nº 2022/00305-5, nº 2023/02680-0 e nº 2023/05171-0, Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP).

Referências

[Alsuliman et al. 2020] Alsuliman, T., Humaidan, D., and Sliman, L. (2020). Machine learning and artificial intelligence in the service of medicine: Necessity or potentiality?

Current research in translational medicine, 68(4):245–251.

- [Ang and Seng 2016] Ang, L.-M. and Seng, K. P. (2016). Big sensor data applications in urban environments. *Big Data Research*, 4:1–12.
- [Bagnall et al. 2017] Bagnall, A., Lines, J., Bostrom, A., Large, J., and Keogh, E. (2017). The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data mining and knowledge discovery*, 31:606–660.
- [Bagnall et al. 2015] Bagnall, A., Lines, J., Hills, J., and Bostrom, A. (2015). Time-series classification with cote: The collective of transformation-based ensembles. *IEEE Transactions on Knowledge and Data Engineering*, 27(9):2522–2535.
- [Barandas et al. 2020] Barandas, M., Folgado, D., Fernandes, L., Santos, S., Abreu, M., Bota, P., Liu, H., Schultz, T., and Gamboa, H. (2020). Tsfel: Time series feature extraction library. *SoftwareX*, 11:100456.
- [Becker 2006] Becker, D. E. (2006). Fundamentals of electrocardiography interpretation. *Anesthesia progress*, 53(2):53–64.
- [Bloomfield 2004] Bloomfield, P. (2004). *Fourier analysis of time series: an introduction*. John Wiley & Sons.
- [Christ et al. 2018] Christ, M., Braun, N., Neuffer, J., and Kempa-Liehr, A. W. (2018). Time series feature extraction on basis of scalable hypothesis tests (tsfresh—a python package). *Neurocomputing*, 307:72–77.
- [Dau et al. 2018] Dau, H. A., Keogh, E., Kamgar, K., Yeh, C.-C. M., Zhu, Y., Gharghabi, S., Ratanamahatana, C. A., Yanping, Hu, B., Begum, N., Bagnall, A., Mueen, A., Batista, G., and Hexagon-ML (2018). The ucr time series classification archive. https://www.cs.ucr.edu/~eamonn/time_series_data_2018/.
- [Deng et al. 2013] Deng, H., Runger, G., Tuv, E., and Vladimir, M. (2013). A time series forest for classification and feature extraction. *Information Sciences*, 239:142–153.
- [Durak and Arikan 2003] Durak, L. and Arikan, O. (2003). Short-time fourier transform: two fundamental properties and an optimal implementation. *IEEE Transactions on Signal Processing*, 51(5):1231–1242.
- [Ebrahimi et al. 2020] Ebrahimi, Z., Loni, M., Daneshtalab, M., and Gharehbaghi, A. (2020). A review on deep learning methods for ecg arrhythmia classification. *Expert Systems with Applications: X*, 7:100033.
- [El-Hajj and Kyriacou 2020] El-Hajj, C. and Kyriacou, P. A. (2020). A review of machine learning techniques in photoplethysmography for the non-invasive cuff-less measurement of blood pressure. *Biomedical Signal Processing and Control*, 58:101870.
- [El Maachi et al. 2020] El Maachi, I., Bilodeau, G.-A., and Bouachir, W. (2020). Deep 1d-convnet for accurate parkinson disease detection and severity prediction from gait. *Expert Systems with Applications*, 143:113075.

- [Faceli et al. 2020] Faceli, K., Lorena, A. C., Gama, J., Almeida, T. A., and Carvalho, A. C. P. L. F. (2020). *Inteligência Artificial—uma abordagem de aprendizado de máquina*. Rio de Janeiro: LTC, 2 edition.
- [Fawaz et al. 2019] Fawaz, H. I., Forestier, G., Weber, J., Idoumghar, L., and Muller, P.-A. (2019). Deep learning for time series classification: a review. *Data Mining and Knowledge Discovery*, 33(4):917–963.
- [Fawaz et al. 2020] Fawaz, H. I., Lucas, B., Forestier, G., Pelletier, C., Schmidt, D. F., Weber, J., Webb, G. I., Idoumghar, L., Muller, P.-A., and Petitjean, F. (2020). InceptionTime: Finding AlexNet for time series classification. *Data Mining and Knowledge Discovery*, 34(6):1936–1962.
- [Foumani et al. 2023] Foumani, N. M., Miller, L., Tan, C. W., Webb, G. I., Forestier, G., and Salehi, M. (2023). Deep learning for time series classification and extrinsic regression: A current survey. *arXiv preprint arXiv:2302.02515*.
- [Fulcher and Jones 2017] Fulcher, B. D. and Jones, N. S. (2017). hctsa: A computational framework for automated time-series phenotyping using massive feature extraction. *Cell systems*, 5(5):527–531.
- [Gay et al. 2021] Gay, D., Bondu, A., Lemaire, V., and Boullé, M. (2021). Interpretable feature construction for time series extrinsic regression. In *Advances in Knowledge Discovery and Data Mining: 25th Pacific-Asia Conference, PAKDD 2021, Virtual Event, May 11–14, 2021, Proceedings, Part I*, pages 804–816. Springer.
- [Guijo-Rubio et al. 2023] Guijo-Rubio, D., Middlehurst, M., Arcencio, G., Silva, D. F., and Bagnall, A. (2023). Unsupervised feature based algorithms for time series extrinsic regression. *arXiv preprint arXiv:2305.01429*.
- [Herrmann et al. 2023] Herrmann, M., Tan, C. W., Salehi, M., and Webb, G. I. (2023). Proximity forest 2.0: A new effective and scalable similarity-based classifier for time series.
- [Hong et al. 2020] Hong, S., Zhou, Y., Shang, J., Xiao, C., and Sun, J. (2020). Opportunities and challenges of deep learning methods for electrocardiogram data: A systematic review. *Computers in biology and medicine*, 122:103801.
- [Hu et al. 2019] Hu, Y., Ji, C., Zhang, Q., Chen, L., Zhan, P., and Li, X. (2019). A novel multi-resolution representation for time series sensor data analysis. *Soft Computing*, pages 1–26.
- [Kashani and Barold 2005] Kashani, A. and Barold, S. S. (2005). Significance of qrs complex duration in patients with heart failure. *Journal of the American College of Cardiology*, 46(12):2183–2192.
- [Köppen 2000] Köppen, M. (2000). The curse of dimensionality. In *World Conference on Soft Computing in Industrial Applications*, volume 1, pages 4–8.

- [Kwapisz et al. 2011] Kwapisz, J. R., Weiss, G. M., and Moore, S. A. (2011). Activity recognition using cell phone accelerometers. *ACM SigKDD Explorations Newsletter*, 12(2):74–82.
- [Lee et al. 2022] Lee, J., Yeom, I., Chung, M. L., Kim, Y., Yoo, S., and Kim, E. (2022). Use of mobile apps for self-care in people with parkinson disease: systematic review. *JMIR mHealth and uHealth*, 10(1):e33944.
- [Lines and Bagnall 2015] Lines, J. and Bagnall, A. (2015). Time series classification with ensembles of elastic distance measures. *Data Mining and Knowledge Discovery*, 29(3):565–592.
- [Lines et al. 2018] Lines, J., Taylor, S., and Bagnall, A. (2018). Time series classification with hive-cote: The hierarchical vote collective of transformation-based ensembles. *ACM Transactions on Knowledge Discovery from Data*, 12(5):1–35.
- [Lubba et al. 2019] Lubba, C. H., Sethi, S. S., Knaute, P., Schultz, S. R., Fulcher, B. D., and Jones, N. S. (2019). catch22: Canonical time-series characteristics: Selected through highly comparative time-series analysis. *Data Mining and Knowledge Discovery*, 33(6):1821–1852.
- [Lucas et al. 2019] Lucas, B., Shifaz, A., Pelletier, C., O’Neill, L., Zaidi, N., Goethals, B., Petitjean, F., and Webb, G. I. (2019). Proximity forest: an effective and scalable distance-based classifier for time series. *Data Mining and Knowledge Discovery*, 33(3):607–635.
- [Martinez-Ríos et al. 2022] Martinez-Ríos, E., Montesinos, L., and Alfaro-Ponce, M. (2022). A machine learning approach for hypertension detection based on photoplethysmography and clinical data. *Computers in Biology and Medicine*, 145:105479.
- [Mazzu-Nascimento et al. 2020] Mazzu-Nascimento, T., de Oliveira Leal, Â. M., Nogueira-de Almeida, C. A., de Avó, L. R. d. S., Carrilho, E., and Silva, D. F. (2020). Noninvasive self-monitoring of blood glucose at your fingertips, literally!: Smartphone-based photoplethysmography. *International Journal of Nutrology*, 13(02):048–052.
- [Middlehurst and Bagnall 2022] Middlehurst, M. and Bagnall, A. (2022). The fresh-prince: A simple transformation based pipeline time series classifier. In *Pattern Recognition and Artificial Intelligence: Third International Conference, ICPRAI 2022, Paris, France, June 1–3, 2022, Proceedings, Part II*, pages 150–161. Springer.
- [Middlehurst et al. 2020] Middlehurst, M., Large, J., and Bagnall, A. (2020). The canonical interval forest (cif) classifier for time series classification. In *2020 IEEE International Conference on Big Data (Big Data)*, pages 188–195.
- [Middlehurst et al. 2021] Middlehurst, M., Large, J., Flynn, M., Lines, J., Bostrom, A., and Bagnall, A. (2021). Hive-cote 2.0: a new meta ensemble for time series classification. *Machine Learning*, 110(11-12):3211–3243.
- [Mitchell 1997] Mitchell, T. M. (1997). Machine learning.

- [Morid et al. 2023] Morid, M. A., Sheng, O. R. L., and Dunbar, J. (2023). Time series prediction using deep learning methods in healthcare. *ACM Transactions on Management Information Systems*, 14(1):1–29.
- [Oguiza 2022] Oguiza, I. (2022). tsai - a state-of-the-art deep learning library for time series and sequential data. Github.
- [Pereira et al. 2020] Pereira, T., Tran, N., Gadhoumi, K., Pelter, M. M., Do, D. H., Lee, R. J., Colorado, R., Meisel, K., and Hu, X. (2020). Photoplethysmography based atrial fibrillation detection: a review. *NPJ digital medicine*, 3(1):3.
- [Perpetuini et al. 2021] Perpetuini, D., Chiarelli, A. M., Cardone, D., Filippini, C., Rinnella, S., Massimino, S., Bianco, F., Bucciarelli, V., Vinciguerra, V., Fallica, P., et al. (2021). Prediction of state anxiety by machine learning applied to photoplethysmography data. *PeerJ*, 9:e10448.
- [Ruiz et al. 2020] Ruiz, A. P., Flynn, M., Large, J., Middlehurst, M., and Bagnall, A. (2020). The great multivariate time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data Mining and Knowledge Discovery*, pages 1–49.
- [Salari et al. 2022] Salari, N., Hosseini-Far, A., Mohammadi, M., Ghasemi, H., Khazaie, H., Daneshkhah, A., and Ahmadi, A. (2022). Detection of sleep apnea using machine learning algorithms based on ecg signals: A comprehensive systematic review. *Expert Systems with Applications*, 187:115950.
- [Schäfer 2015] Schäfer, P. (2015). The boss is concerned with time series classification in the presence of noise. *Data Mining and Knowledge Discovery*, 29(6):1505–1530.
- [Schäfer and Höggqvist 2012] Schäfer, P. and Höggqvist, M. (2012). Sfa: a symbolic fourier approximation and index for similarity search in high dimensional datasets. In *EDBT '12: Proceedings of the 15th International Conference on Extending Database Technology*, pages 516–527.
- [Silva 2017] Silva, D. F. (2017). *Large-Scale Similarity-Based Time Series Mining*. PhD thesis, Universidade de São Paulo, São Carlos.
- [Silva et al. 2023] Silva, D. F., de M. Júnior, J. G. B., Domingues, L. V., and Mazzu-Nascimento, T. (2023). Hemoglobin estimation from smartphone-based photoplethysmography with small data. In *Computer-Based Medical Systems*, page No prelo. IEEE.
- [Silva et al. 2018] Silva, D. F., Giusti, R., Keogh, E., and Batista, G. E. (2018). Speeding up similarity search under dynamic time warping by pruning unpromising alignments. *Data Mining and Knowledge Discovery*, 32:988–1016.
- [Tan et al. 2020] Tan, C. W., Bergmei, C., Petitjean, F., Schmidt, D., Webb, G. I., Bagnall, A., and Keogh, E. (2020). The monash, uea & ucr time series extrinsic regression archive. <http://tseregression.org/>.

- [Tan et al. 2021] Tan, C. W., Bergmeir, C., Petitjean, F., and Webb, G. I. (2021). Time series extrinsic regression. *Data Mining and Knowledge Discovery*, pages 1–29.
- [Thoms et al. 2017] Thoms, L.-J., Colicchia, G., and Girwidz, R. (2017). Phonocardiography with a smartphone. *Physics Education*, 52(2):023004.
- [WHO 2011] WHO, W. H. O. (2011). mhealth: new horizons for health through mobile technologies. *mHealth: new horizons for health through mobile technologies*.
- [Wolpert 2002] Wolpert, D. H. (2002). The supervised learning no-free-lunch theorems. *Soft computing and industry*, pages 25–42.
- [Yeh et al. 2018] Yeh, C.-C. M., Zhu, Y., Ulanova, L., Begum, N., Ding, Y., Dau, H. A., Zimmerman, Z., Silva, D. F., Mueen, A., and Keogh, E. (2018). Time series joins, motifs, discords and shapelets: a unifying view that exploits the matrix profile. *Data Mining and Knowledge Discovery*, 32(1):83–123.

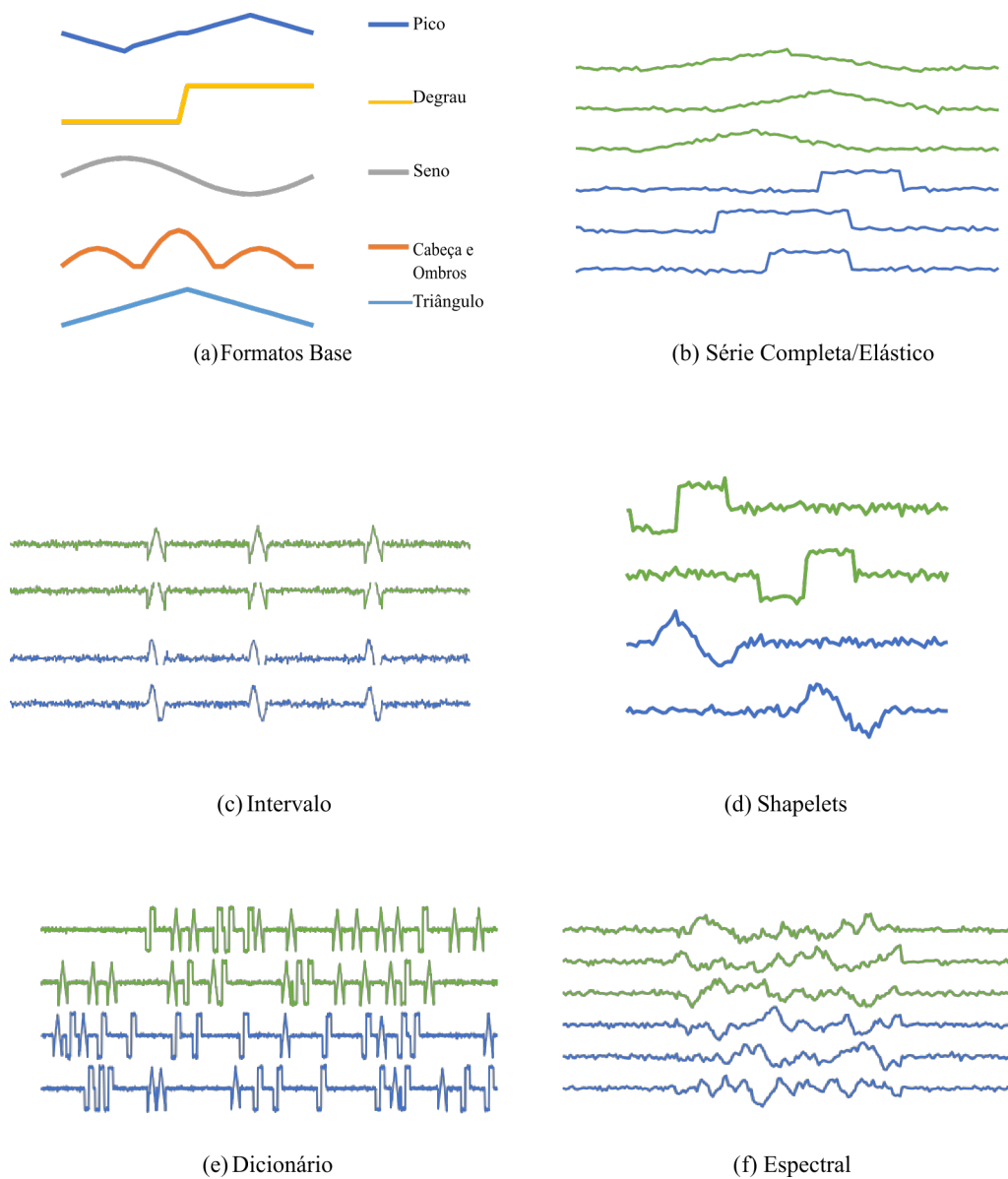


Figura 2.16. Ilustração dos domínios considerados pelos modelos do HIVE-COTE. Adaptado de [Lines et al. 2018].

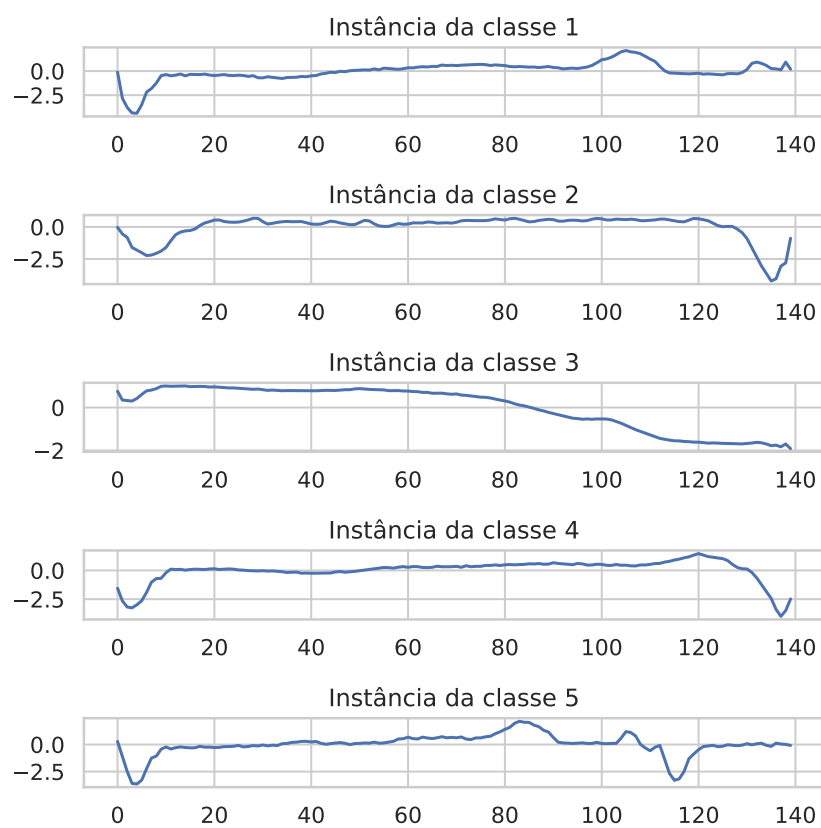


Figura 2.17. Exemplos de instâncias contidas em cada classe do conjunto de dados ECG5000.

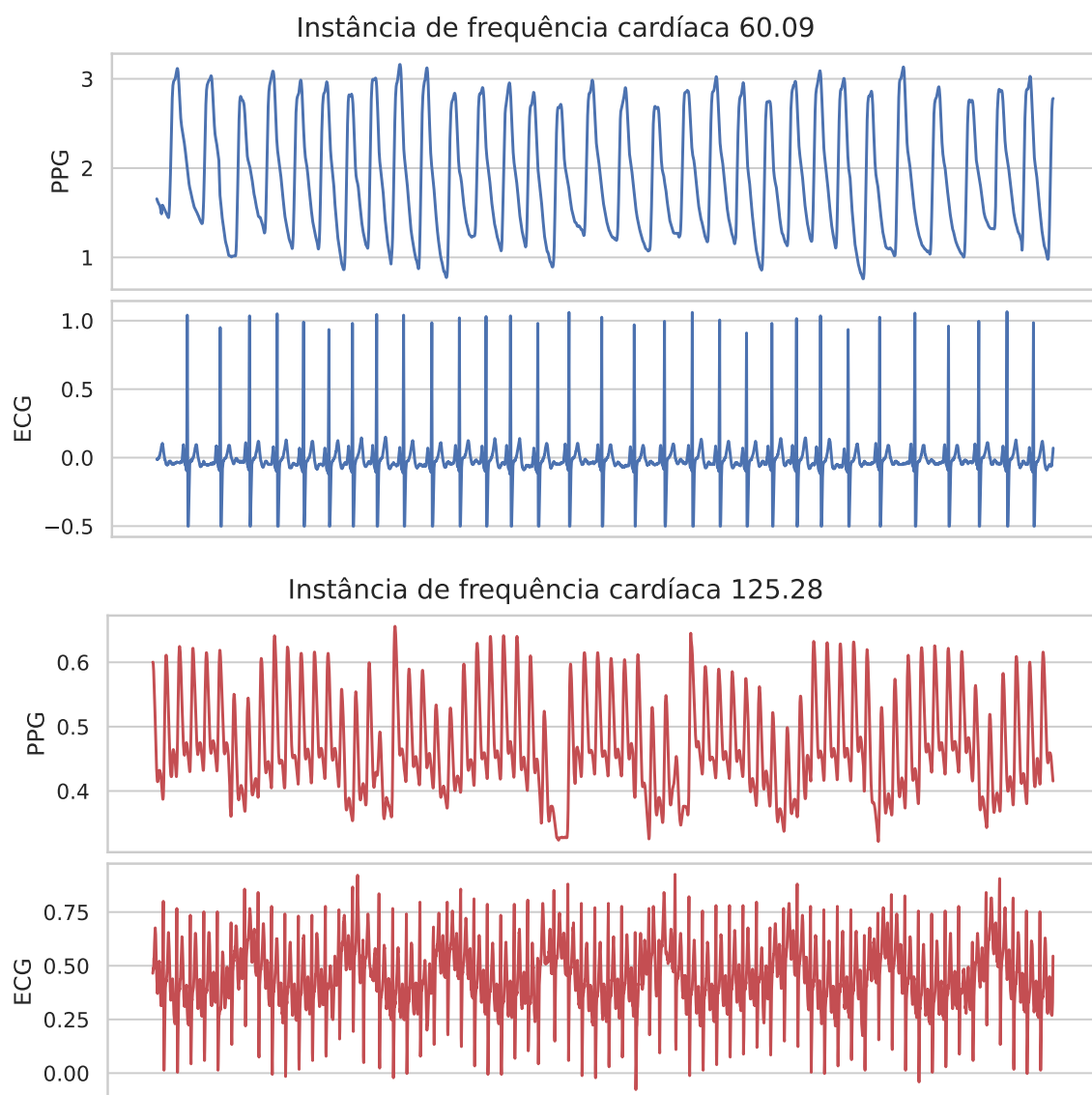


Figura 2.18. Exemplos de instâncias do conjunto de dados BIDMCHR com suas respectivas frequências cardíacas.

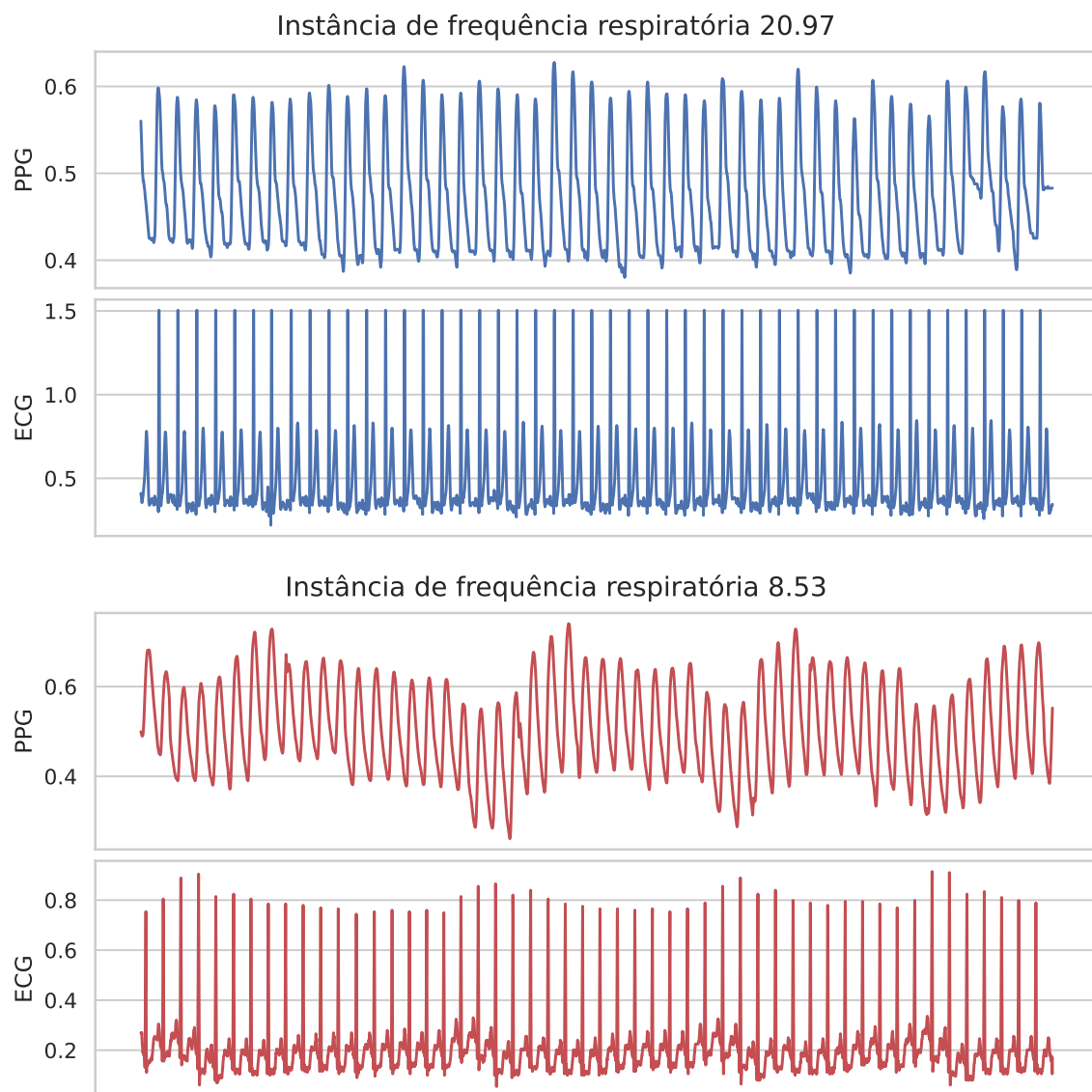


Figura 2.19. Exemplos de instâncias do conjunto de dados BIDMCRR com suas respectivas frequências respiratórias.

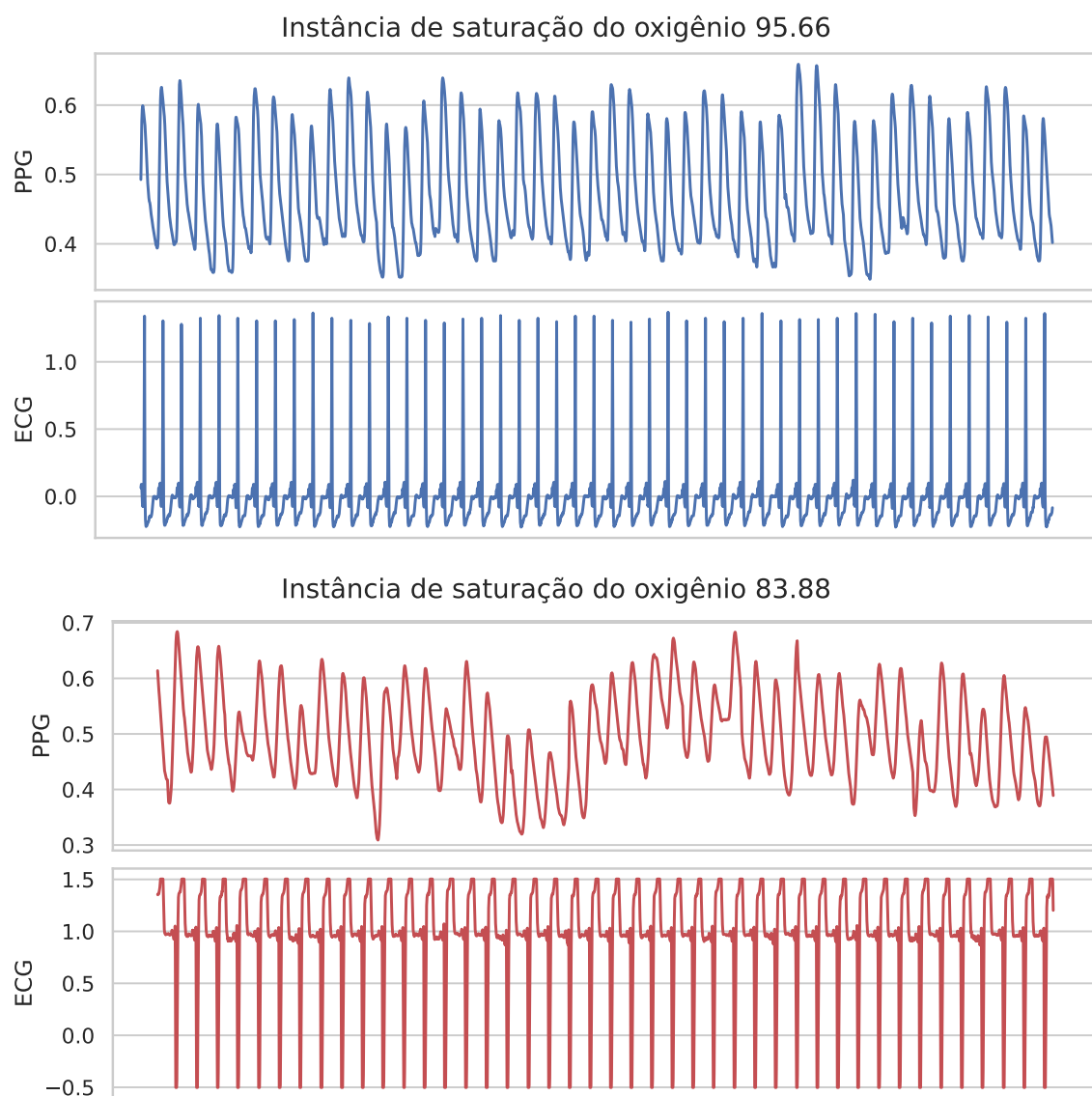


Figura 2.20. Exemplos de instâncias do conjunto de dados BIDMCSpO2 com suas respectivas saturações do oxigênio.

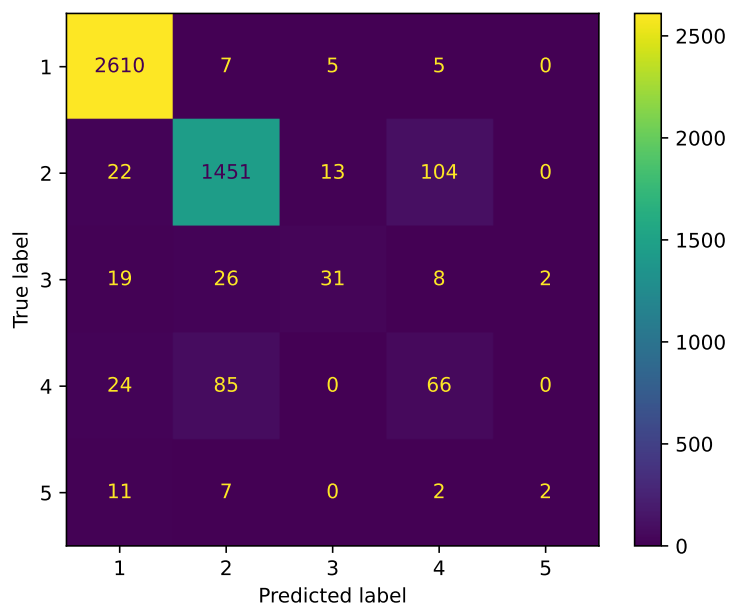


Figura 2.21. Matriz de Confusão gerada pelo Algoritmo KNN.

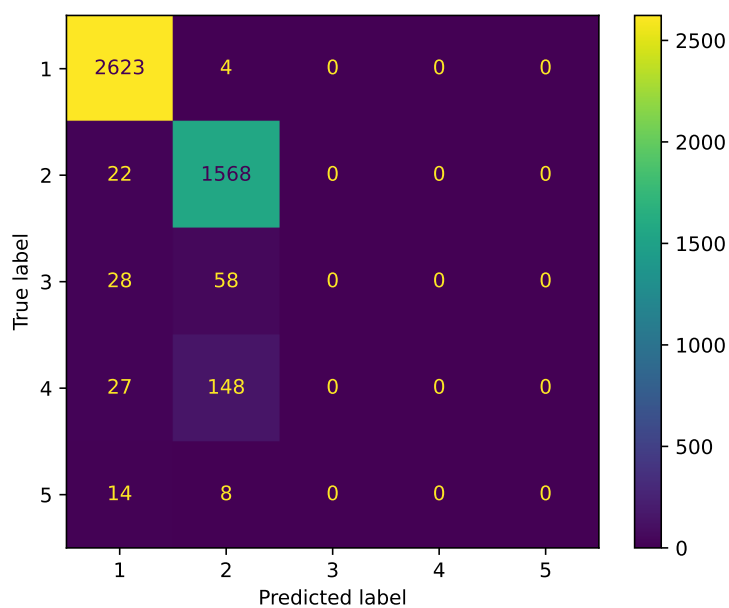


Figura 2.22. Matriz de Confusão gerada pelo InceptionTime após treinamento no conjunto de dados ECG5000.